

PDD – PROCESS DEFINITION DOCUMENT (PDD)

Disciplina: Técnicas de Hyperautomation

Professor: Moisés Levy

Aluno: Huan Cruz de Oliveira

Data: 10/07/2026

Semana: 04

Versão do Documento: 1.0

Nome do Processo: Cadastro de Usuários – Portal Fake

1. Descrição do processo

O Portal Fake é um sistema de automação desenvolvido para simular um ambiente corporativo de desenvolvimento colaborativo. O projeto utiliza Git como sistema de controle de versão distribuído, GitHub como plataforma de hospedagem do código-fonte e GitFlow como estratégia de gerenciamento das branches. Cada integrante da equipe desenvolve uma funcionalidade específica em uma branch independente, garantindo isolamento das alterações, rastreabilidade das modificações e integração controlada por meio de Pull Requests e revisões de código.

2. Objetivo do Processo

Objetivo Geral

Implementar um fluxo padronizado de desenvolvimento utilizando GitFlow, garantindo organização, rastreabilidade, colaboração e qualidade durante todo o ciclo de desenvolvimento.

Objetivos específicos

- Organizar o desenvolvimento em branches.
- Controlar versões.
- Evitar conflitos.
- Facilitar integração.
- Melhorar qualidade.
- Garantir revisão de código.

3. Fluxo do Processo (Resumo)

O processo de desenvolvimento do **Portal Fake** utilizando GitFlow é composto por uma sequência organizada de atividades que garantem o desenvolvimento colaborativo, o controle de versões e a integração segura das funcionalidades implementadas.

O fluxo do processo ocorre conforme descrito a seguir:

- Criação do repositório do projeto **Portal Fake** no GitHub.
- Inicialização da branch **main**, destinada à versão estável da aplicação.
- Criação da branch **develop**, utilizada para integração contínua das funcionalidades desenvolvidas.
- Criação das **feature branches** para cada módulo do sistema, como

`feature/cadastro`, `feature/consulta` e `feature/exportacao`.

- Desenvolvimento das funcionalidades em ambiente local utilizando Python e Visual Studio Code.
- Realização de testes locais para validação das funcionalidades implementadas.

- Registro das alterações utilizando os comandos `git add` e `git commit`.
- Envio das alterações para o repositório remoto utilizando o comando `git push`.
- Abertura de um **Pull Request (PR)** para solicitar a integração da feature branch à branch **develop**.
- Revisão do código (Code Review) por outro integrante da equipe ou responsável técnico.
- Correção das inconsistências identificadas durante a revisão, quando necessário.
- Aprovação do Pull Request e realização do **merge** da feature branch na branch **develop**.
- Execução de testes integrados para validação da aplicação após a integração das funcionalidades.
- Realização do **merge** da branch **develop** na branch **main**, disponibilizando a nova versão estável do Portal Fake.
- Atualização do histórico do projeto no GitHub e encerramento do ciclo de desenvolvimento da funcionalidade.

4. Regras de Negócio

As regras de negócio descritas a seguir estabelecem as diretrizes para o desenvolvimento colaborativo do Portal Fake utilizando Git, GitHub e GitFlow, garantindo organização, rastreabilidade e qualidade do código-fonte.

ID	Regra de Negócio
RN01	É proibido realizar alterações diretamente na branch main , pois ela representa a versão estável da aplicação.
RN02	Todo desenvolvimento de uma nova funcionalidade deve ser realizado em uma feature branch específica.
RN03	Cada integrante da equipe é responsável pelo desenvolvimento e manutenção da sua própria feature branch.
RN04	Toda alteração realizada no código-fonte deve ser registrada por meio de commits com mensagens claras e padronizadas.
RN05	Após concluir uma funcionalidade, o desenvolvedor deve enviar sua branch para o GitHub utilizando o comando git push .
RN06	A integração de uma feature branch à branch develop somente poderá ocorrer por meio de um Pull Request (PR) .
RN07	Todo Pull Request deve passar por revisão de código (Code Review) antes de ser aprovado.

ID	Regra de Negócio
RN08	O merge da feature branch somente poderá ser realizado após a aprovação do Pull Request.
RN09	Caso sejam identificados erros durante a revisão ou nos testes, as correções deverão ser realizadas na própria feature branch , sendo necessário submeter um novo Pull Request.
RN10	O merge da branch develop para a branch main somente poderá ocorrer após a validação de todas as funcionalidades e aprovação da equipe responsável.

5. Exceções

- Pull Request rejeitado durante a revisão.
- Conflito de merge entre branches.
- Erros encontrados durante os testes locais.
- Falhas identificadas nos testes integrados.
- Problemas de sincronização entre repositório local e remoto.
- Commits enviados para a branch incorreta.
- Falha de conexão com a internet durante o envio do código para o GitHub.
- Branch de funcionalidade criada a partir da branch incorreta.
- Exclusão acidental de arquivos importantes do projeto.
- Alterações simultâneas no mesmo arquivo, gerando conflitos de código

6. Pré-condições

Antes do início do processo é necessário que:

- Git esteja instalado e configurado na máquina do desenvolvedor;
- GitHub esteja configurado com uma conta ativa;
- Repositório do projeto criado no GitHub;
- Branch **main** criada e inicializada;
- Branch **develop** criada a partir da **main**;
- Equipe de desenvolvimento definida;
- Cada integrante tenha acesso ao repositório;
- Permissões de leitura e escrita concedidas aos colaboradores;
- Visual Studio Code instalado;
- Python instalado e configurado;

7. Ambiente de Desenvolvimento

O desenvolvimento do Portal Fake é realizado em um ambiente colaborativo utilizando ferramentas de controle de versão, hospedagem de código e desenvolvimento de software, permitindo que a equipe trabalhe de forma organizada e segura.

Ferramentas Utilizadas

- **Git:** sistema de controle de versão distribuído para gerenciamento do código-fonte.
- **GitHub:** plataforma para hospedagem do repositório, gerenciamento de branches, Pull Requests e colaboração da equipe.
- **Visual Studio Code (VS Code):** editor de código utilizado no desenvolvimento das funcionalidades.

- **Python:** linguagem de programação utilizada na implementação dos módulos do sistema.

Estrutura do Repositório

- **main** → versão estável da aplicação.
- **develop** → integração das funcionalidades.
- **feature/cadastro** → módulo de cadastro.
- **feature/consulta** → módulo de consulta.
- **feature/exportacao** → módulo de exportação.

Recursos Utilizados

- Controle de versão com Git.
- Organização do desenvolvimento por meio do GitFlow.
- Integração das funcionalidades utilizando Pull Requests.
- Revisão de código antes da realização do merge.
- Testes das funcionalidades antes da disponibilização da versão final.

8. Estrutura do Processo (Resumo Técnico)

O processo de desenvolvimento segue o modelo GitFlow:

- Versionamento utilizando Git;
- Organização em branches;
- Desenvolvimento independente por funcionalidade;
- Registro das alterações com commits;
- Envio para o GitHub através do git push;
- Integração utilizando Pull Requests;
- Revisão de código (Code Review);
- Merge controlado para develop;
- Testes integrados;
- Merge final para a branch main, disponibilizando a versão estável da aplicação.